

# SPARQL Queries and Updates

Simon Frost

## Overview

SPARQL is the standard query language for RDF data. RDFLib.jl includes a built-in SPARQL engine supporting SELECT, CONSTRUCT, ASK, DESCRIBE, and UPDATE operations.

```
using RDFLib

# Build a small knowledge graph about scientists
g = RDFGraph()
ex = Namespace("http://example.org/")
bind!(g, "ex", ex)
bind!(g, "foaf", Namespace("http://xmlns.com/foaf/0.1/"))

scientists = [
    ("einstein", "Albert Einstein", 1879, "physics", "relativity"),
    ("curie", "Marie Curie", 1867, "chemistry", "radioactivity"),
    ("darwin", "Charles Darwin", 1809, "biology", "evolution"),
    ("newton", "Isaac Newton", 1643, "physics", "gravity"),
    ("turing", "Alan Turing", 1912, "computer_science", "computation"),
    ("lovelace", "Ada Lovelace", 1815, "computer_science", "programming"),
]

for (id, name, born, field, topic) in scientists
    s = ex(id)
    add!(g, Triple(s, RDF.type, FOAF("Person")))
    add!(g, Triple(s, FOAF("name"), Literal(name)))
    add!(g, Triple(s, ex("birthYear"), Literal(born)))
    add!(g, Triple(s, ex("field"), ex(field)))
    add!(g, Triple(s, ex("knownFor"), ex(topic)))
end

# Add some relationships
```

```

add!(g, Triple(ex("curie"), FOAF("knows"), ex("einstein")))
add!(g, Triple(ex("newton"), ex("influenced"), ex("einstein")))
add!(g, Triple(ex("lovelace"), ex("influenced"), ex("turing")))

println("Graph has $(length(g)) triples")

```

Graph has 33 triples

## SELECT Queries

Retrieve tabular results:

```

results = sparql_query(g, """
    PREFIX foaf: <http://xmlns.com/foaf/0.1/>
    PREFIX ex: <http://example.org/>

    SELECT ?name ?field WHERE {
        ?person a foaf:Person .
        ?person foaf:name ?name .
        ?person ex:field ?field .
    }
    ORDER BY ?name
""")

for row in results
    println("$(row["name"]) - $(row["field"])")
end

```

```

Literal("Ada Lovelace") - URIRef("http://example.org/computer_science")
Literal("Alan Turing") - URIRef("http://example.org/computer_science")
Literal("Albert Einstein") - URIRef("http://example.org/physics")
Literal("Charles Darwin") - URIRef("http://example.org/biology")
Literal("Isaac Newton") - URIRef("http://example.org/physics")
Literal("Marie Curie") - URIRef("http://example.org/chemistry")

```

## Filtering

```

# Scientists born before 1850
results = sparql_query(g, """
    PREFIX foaf: <http://xmlns.com/foaf/0.1/>
    PREFIX ex: <http://example.org/>

    SELECT ?name ?year WHERE {
        ?person foaf:name ?name .
        ?person ex:birthYear ?year .
        FILTER (?year < 1850)
    }
    ORDER BY ?year
""")

println("Scientists born before 1850:")
for row in results
    println("  $(row["name"]) ($(row["year"]))")
end

```

Scientists born before 1850:

```

    Literal("Isaac Newton") (Literal("1643", datatype=URIRef("http://www.w3.org/2001/XMLSchema#integer")))
    Literal("Charles Darwin") (Literal("1809", datatype=URIRef("http://www.w3.org/2001/XMLSchema#integer")))
    Literal("Ada Lovelace") (Literal("1815", datatype=URIRef("http://www.w3.org/2001/XMLSchema#integer")))

```

## OPTIONAL and BOUND

```

# Find scientists and their influences (if any)
results = sparql_query(g, """
    PREFIX foaf: <http://xmlns.com/foaf/0.1/>
    PREFIX ex: <http://example.org/>

    SELECT ?name ?influenced WHERE {
        ?person foaf:name ?name .
        OPTIONAL {
            ?person ex:influenced ?other .
            ?other foaf:name ?influenced .
        }
    }
    ORDER BY ?name
""")

```

```

for row in results
  infl = get(row, "influenced", nothing)
  if infl != nothing
    println("${row["name"]} influenced ${infl}")
  else
    println("${row["name"]}")
  end
end
end

```

```

Literal("Ada Lovelace") influenced Literal("Alan Turing")
Alan Turing
Albert Einstein
Charles Darwin
Literal("Isaac Newton") influenced Literal("Albert Einstein")
Marie Curie

```

## Aggregation

```

# Count scientists per field
results = sparql_query(g, """
  PREFIX ex: <http://example.org/>

  SELECT ?field (COUNT(?person) AS ?count) WHERE {
    ?person ex:field ?field .
  }
  GROUP BY ?field
  ORDER BY DESC(?count)
""")

println("Scientists per field:")
for row in results
  println("  ${row["field"]}: ${row["count"]}")
end

```

Scientists per field:

```

URIRef("http://example.org/computer_science"): Literal("2", datatype=URIRef("http://www.w3.org/2001/XMLSchema#integer"))
URIRef("http://example.org/physics"): Literal("2", datatype=URIRef("http://www.w3.org/2001/XMLSchema#integer"))
URIRef("http://example.org/chemistry"): Literal("1", datatype=URIRef("http://www.w3.org/2001/XMLSchema#integer"))
URIRef("http://example.org/biology"): Literal("1", datatype=URIRef("http://www.w3.org/2001/XMLSchema#integer"))

```

## ASK Queries

Test whether a pattern exists:

```
# Is there a physicist in the graph?
result = sparql_query(g, """
    PREFIX ex: <http://example.org/>
    ASK { ?person ex:field ex:physics }
    """)
println("Is there a physicist? ", result)
```

Is there a physicist? true

## CONSTRUCT Queries

Build a new graph from query results:

```
# Extract a subgraph of physicists
physicists = sparql_query(g, """
    PREFIX foaf: <http://xmlns.com/foaf/0.1/>
    PREFIX ex: <http://example.org/>

    CONSTRUCT {
        ?person a foaf:Person .
        ?person foaf:name ?name .
        ?person ex:knownFor ?topic .
    } WHERE {
        ?person ex:field ex:physics .
        ?person foaf:name ?name .
        ?person ex:knownFor ?topic .
    }
    """)

println("Physicist subgraph ($(length(physicists)) triples):")
println(serialize(physicists, TurtleFormat()))
```

```
Physicist subgraph (6 triples):
@prefix ns1: <http://example.org/> .
@prefix ns2: <http://xmlns.com/foaf/0.1/> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
```

```
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix skos: <http://www.w3.org/2004/02/skos/core#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
```

```
ns1:einstein a ns2:Person ;
    ns1:knownFor ns1:relativity ;
    ns2:name "Albert Einstein" .
```

```
ns1:newton a ns2:Person ;
    ns1:knownFor ns1:gravity ;
    ns2:name "Isaac Newton" .
```

## DESCRIBE

Get a description of a resource:

```
description = sparql_query(g, """
    PREFIX ex: <http://example.org/>
    DESCRIBE ex:einstein
    """)

println("Description of Einstein ($(length(description)) triples):")
for t in description
    println("  ", t.predicate, " → ", t.object)
end
```

Description of Einstein (5 triples):

```
URIRef("http://www.w3.org/1999/02/22-rdf-syntax-ns#type") → URIRef("http://xmlns.com/foaf/")
URIRef("http://xmlns.com/foaf/0.1/name") → Literal("Albert Einstein")
URIRef("http://example.org/birthYear") → Literal("1879", datatype=URIRef("http://www.w3.org/"))
URIRef("http://example.org/field") → URIRef("http://example.org/physics")
URIRef("http://example.org/knownFor") → URIRef("http://example.org/relativity")
```

## SPARQL UPDATE

Modify the graph with INSERT and DELETE:

```
# Add a new fact
sparql_update(g, """
    PREFIX ex: <http://example.org/>
    INSERT DATA {
```

```

        ex:einstein ex:nationality "German" .
        ex:curie ex:nationality "Polish" .
    }
    """)

# Verify
results = sparql_query(g, """
    PREFIX foaf: <http://xmlns.com/foaf/0.1/>
    PREFIX ex: <http://example.org/>
    SELECT ?name ?nationality WHERE {
        ?person foaf:name ?name .
        ?person ex:nationality ?nationality .
    }
    """)
for row in results
    println("${row["name"]}: ${row["nationality"]}")
end

```

```

Literal("Marie Curie"): Literal("Polish")
Literal("Albert Einstein"): Literal("German")

```

```

# DELETE/INSERT (modify existing data)
sparql_update(g, """
    PREFIX ex: <http://example.org/>
    DELETE { ex:einstein ex:nationality "German" }
    INSERT { ex:einstein ex:nationality "German-American" }
    WHERE { ex:einstein ex:nationality "German" }
    """)

results = sparql_query(g, """
    PREFIX ex: <http://example.org/>
    SELECT ?nat WHERE { ex:einstein ex:nationality ?nat }
    """)
println("Updated nationality: ", results[1]["nat"])

```

```

Updated nationality: Literal("German-American")

```

## String Functions

```

results = sparql_query(g, """
    PREFIX foaf: <http://xmlns.com/foaf/0.1/>

    SELECT ?name WHERE {
        ?person foaf:name ?name .
        FILTER (CONTAINS(?name, "a") || CONTAINS(?name, "A"))
    }
    ORDER BY ?name
""")

println("Names containing 'a':")
for row in results
    println("  ", row["name"])
end

```

```

Names containing 'a':
  Literal("Ada Lovelace")
  Literal("Alan Turing")
  Literal("Albert Einstein")
  Literal("Charles Darwin")
  Literal("Isaac Newton")
  Literal("Marie Curie")

```

## UNION

```

results = sparql_query(g, """
    PREFIX foaf: <http://xmlns.com/foaf/0.1/>
    PREFIX ex: <http://example.org/>

    SELECT ?name ?relation WHERE {
        {
            ?person ex:influenced ?other .
            ?person foaf:name ?name .
            BIND("influenced someone" AS ?relation)
        }
        UNION
        {
            ?person foaf:knows ?other .
            ?person foaf:name ?name .
        }
    }

```



```

        BIND("knows someone" AS ?relation)
    }
}
""")

for row in results
    println("${row["name"]} ${row["relation"]}")
end

```

```

Literal("Ada Lovelace") Literal("influenced someone")
Literal("Isaac Newton") Literal("influenced someone")
Literal("Marie Curie") Literal("knows someone")

```

## VALUES (Inline Data)

```

results = sparql_query(g, ""
    PREFIX foaf: <http://xmlns.com/foaf/0.1/>
    PREFIX ex: <http://example.org/>

    SELECT ?name ?field WHERE {
        VALUES ?field { ex:physics ex:computer_science }
        ?person ex:field ?field .
        ?person foaf:name ?name .
    }
""")

println("Physicists and Computer Scientists:")
for row in results
    println("  ${row["name"]}")
end

```

```

Physicists and Computer Scientists:
  Literal("Isaac Newton")
  Literal("Albert Einstein")
  Literal("Ada Lovelace")
  Literal("Alan Turing")

```

## SPARQL Results Serialization

Query results can be serialized to standard formats:

```
results = sparql_query(g, """
    PREFIX foaf: <http://xmlns.com/foaf/0.1/>
    SELECT ?name WHERE { ?p foaf:name ?name } ORDER BY ?name LIMIT 3
""")

# JSON format
json_results = sparql_results_json(results)
println("JSON Results:")
println(json_results)
```

JSON Results:

```
{"head":{"vars":["name"]},"results":{"bindings":[{"name":{"type":"literal","value":"Ada Love
```

```
# CSV format
csv_results = sparql_results_csv(results)
println("CSV Results:")
println(csv_results)
```

CSV Results:

```
name
Ada Lovelace
Alan Turing
Albert Einstein
```